

Pré-Projeto — PratoFácil

Sistemas Distribuídos e Mobile — Avaliação A3

Documento da fase de descoberta (relatório inicial)

I. Relatório Inicial (Fase de Descoberta)

1. Definição da Problemática Central

Pequenos empreendedores que produzem e entregam alimentos de nicho muitas vezes gerenciam suas vendas de forma manual (anotando em cadernos ou via WhatsApp). Isso gera perda de informações, erros nos pedidos e dificulta a escalabilidade do negócio. O problema central é a falta de um sistema simples, centralizado e distribuído para gestão de ponta a ponta desses pedidos.

2. Público-alvo

Microempreendedores individuais (vendedores autônomos de alimentos locais) e seus respectivos clientes finais.

3. Requisitos Iniciais (Escopo Reduzido)

- **RF01:** O sistema deve permitir a listagem do cardápio disponível.
- **RF02:** O cliente deve poder realizar um pedido via requisição HTTP.
- **RF03:** O empreendedor deve poder atualizar o status do pedido (ex.: “Em preparo”, “Saiu para entrega”, “Entregue”).

Nota: Manteremos o foco unicamente no ciclo de vida do pedido para garantir a excelência da implementação.

II. O Software em Si (Arquitetura e Spring Boot)

Para aplicar os conceitos de Sistemas Distribuídos, o software não será um monólito isolado rodando em uma única máquina, mas sim uma **API RESTful**.

- **Tecnologia Principal:** Java com Spring Boot (Spring Web, Spring Data JPA).
- **Arquitetura:** Padrão MVC/API REST. O Spring Boot atuará como o **Middleware** (fornecendo os serviços via webservice) entre o cliente (uma interface web simples ou o Postman para testes) e o Banco de Dados.
- **Banco de Dados Distribuído / Nuvem:** Para demonstrar conhecimento em nuvem e banco de dados, hospedar o banco relacional em uma infraestrutura *Cloud* (como um Oracle Database ou RDS na AWS).
- **Transparência:** O cliente (front-end) faz uma requisição HTTP para a API e não faz ideia de onde o servidor está hospedado, em qual linguagem foi escrito ou onde o banco de dados armazena as informações (Transparência de Localização e Acesso).

III. Relatório Final e Documentação (Template do Artigo)

Estrutura das seções essenciais para cobrir todos os tópicos da disciplina (a ser adaptada ao template oficial fornecido pela instituição):

Título: Desenvolvimento de uma API REST para Gestão de Pedidos Locais: Uma Abordagem Prática em Sistemas Distribuídos

1. Introdução

- Apresentação do contexto (a realidade dos pequenos empreendedores).
- O objetivo do artigo e do software desenvolvido.

2. Fundamentação Teórica e Conceitos Aplicados — “amarra” a teoria da sala de aula com o código:

- **Sistemas Distribuídos e Transparência:** como a API abstrai a complexidade do banco de dados para o usuário final.
- **Middleware e Webservices:** como o Spring Boot age como o middleware da aplicação, recebendo requisições, processando regras de negócio e acessando os dados.
- **Protocolo HTTP e API REST:** como o sistema se comunica (verbos GET para listar o cardápio, POST para criar pedidos, PUT/PATCH para atualizar status).
- **Serviços em Nuvem e Banco de Dados:** a escolha do banco de dados e as vantagens de mantê-lo em um servidor em nuvem separado da aplicação.

3. Metodologia e Desenvolvimento

- Quais tecnologias foram usadas (Java, Spring Boot, Maven, Banco de Dados escolhido).
- Diagrama simples da arquitetura (Cliente → API Spring Boot → Banco de Dados).

4. Documentação de Testes de Usabilidade e API

- **Métodos:** como a API foi testada (ex.: Postman ou Swagger para simular o cliente final).
- **Dados coletados:** tempo de resposta das requisições, retornos de erro tratados (ex.: HTTP 404 Not Found ao atualizar um pedido inexistente).
- **Ajustes feitos:** problemas encontrados durante o desenvolvimento (ex.: CORS ou serialização de JSON) e como foram resolvidos.

5. Considerações Finais

- Reflexão sobre como a arquitetura distribuída facilita a escalabilidade do negócio de entregas.